jigsaw-agile-community-rules

1位 Solution

Rank	1
Team	Guanshuo Xu
Author	Guanshuo Xu
Topic ID	613305
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613305
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

コードへのリンクはこちらです。

この興味深いコンペを企画・開催してくださった Kaggle と Jigsaw に感謝します。LLM のファインチューニングの実践的な側面を探る、素晴らしい機会でした。

検証戦略

分かっていること:

- train.csv には、コメント（body）付きの2つのルールが含まれている。
- test.csv には6つのルールが含まれ、そのうち4つは新しいルールである。
- 4つの新しいルールは、すでに特定されている。

しかし、強いローカル検証を構築するには、4つの新しいルールについて現実的なコメントを生成する必要があり、これは簡単な作業ではありません。

以下の理由から、Public LB の方が検証には良い選択だと考えました。

- ホストは、public/private LB の分割はランダムだと述べている。つまり、public LB は private LB の不偏推定量として機能する。
- テストデータ全体のサイズは小さくなく、public LB はその約30%を占める。これにより、スコア推定の分散が小さくなる。

そのため、すべてのモデルはオンラインでファインチューニングし、public LB で検証しました。

データ処理

このnotebookは、データ処理とファインチューニングの優れたベースラインを提供しています。アイデアは、test.csv にある positive/negative example を学習データとして使うことです。データ処理パイプラインに加えた主な変更は次のとおりです。

- LLM に組み込まれた chat template を使用した。
- 重複除去の前に subreddit 列を除外した。subreddit を含めると、異なる subreddit をまたいで不要な重複が生じうるためである。この変更後の性能向上は大きかったです！

ファインチューニング

当初は素の transformers + pytorch のコードを使っていました。これは非常にうまく動き、Qwen3-4b-instruct-2507、llama3.2-3b、phi-4-mini-instruct のアンサンブルで早い段階で public LB 0.931 を達成しました。しかし、メモリの制約で 7B を超えるモデルにはスケールできませんでした。その後 Unsloth に切り替えました。こちらははるかにメモリ効率が良く、14B モデルを 16GB の GPU メモリに容易に収められます。 ファインチューニングの前に、test.csv 由来の例を1回アップサンプリングして重みを増やし、その後1 epoch ファインチューニングしました。これは、train.csv について1 epoch、test.csv について2 epoch に相当します。 chat template は、"Yes" や "No" のターゲットの前後に不要なトークンを入れることがあります。たとえば Qwen3 モデルでの "\n\n\n\n" や、"Yes" や "No" のターゲットの後に続く1つ以上のトークンです。デフォルトでは、ファインチューニングはこれらの余分なトークンにも loss をかけますが、これは不要で、学習を遅くする可能性もあります。 これに対処するため、これらのトークンを手作業で loss 計算から除外し、"Yes" または "No" のトークンだけが loss に寄与するようにしました。このステップの追加の利点として、異なるモデル間で学習の挙動が標準化され、収束速度がより一貫するため、すべてのモデルで1組のハイパーパラメータを使えるようになりました。

推論

最後の線形層は語彙全体に写像するので、Yes/No トークンでファインチューニングした後でも、確率が他のトークンに散らばる確率は無視できないほどありえます。そこで、多くのバリアントについて最初のトークンの候補集合を作りました。

- ["Yes","YES","Y","yes","True"] -> Yes_ids
- ["No","NO","N","no","False"] -> No_ids
- さらに、それらのスペース付きの形。

これらのバリアントは、予測されるトークンの topK のうち最も頻度の高いものを出力するローカル実験で得ました。 スコアリングでは、Yes_ids 対 No_ids の log-odds の sigmoid に相当する確率を出力しました。

推論を高速化するその他の工夫:

- テストデータを長さでソートする
- forward() のみを使う。必要なのは最後のトークンの logits だけなので、生成時のその他すべての処理を避けられる。

後処理

各モデルの出力について、ルールごとの順位を計算し、スコアを [0, 1] の範囲に正規化し、正規化した値をもとにアンサンブルしました。この後処理ステップは一貫した改善をもたらし、初期のテストで public LB スコアを 0.929 から 0.931 に上げました。

モデルの性能と最終アンサンブル

- 一般に、大きいモデルほど性能が良かった。
- Qwen3 は他の一般的な LLM より優れていた。
- 最終アンサンブルは、これら6モデルの加重平均である。

model	Public LB	Private LB
Qwen3-14b	0.9297	0.9239
Qwen2.5-14b	0.9287	0.9232
Qwen3-8b	0.9272	0.9236
Qwen3-4b-instruct-2507	0.9258	0.9198
llama3.1-8b	0.9257	0.9202
Ettin-400M	0.8991	0.8944
ensemble	0.9344	0.9293

補足Q&A

CPMP — 質問

解法の共有とおめでとうございます。

私は ettin を試しましたが、自分のテストでは deberta v3 より悪かったです。deberta v3 は試しましたか？

> 各モデルの出力について、ルールごとの順位を計算し、スコアを [0, 1] の範囲に正規化し、正規化した値をもとにアンサンブルしました。この後処理ステップは一貫した改善をもたらし、初期のテストで public LB スコアを 0.929 から 0.931 に上げました。

先にルールでグループ化せずに rank average するというミスをしたなんて信じられません。お恥ずかしい。

あなたの notebook のモデルは private になっています。何か理由がありますか？ 理由があるならそれで構いませんが、一応聞いてみようと思いました。

Guanshuo Xu — 回答

優勝した実行に必要なモデルはすべて public にしておくべきでした。

私の場合、Ettin と Modernbert はどちらも Deberta より良かったのですが、それらには力を入れておらず、アンサンブルを豊かにするためだけに使いました。

Chris Deotte — 質問

おめでとうございます、Guanshuo、素晴らしい仕事です！

12時間の提出の間に、6つの LLM モデルそれぞれを 100% test + 100% train のデータで1 epoch ファインチューニングしたのですか？ 2xT4 GPU で12時間以内にそれを達成するのは見事です。

Guanshuo Xu — 回答

テストデータはアップサンプリングしたので、各モデルは 200% test + 100% train のデータでファインチューニングしています。

Chris Deotte — 質問

それはすごい。Unsloth はとても速いのでしょうね。提出中に Llama3.2-3B を学習する public notebook は、5% の test データと 100% の train データで学習します。その後テストを推論します。それで6時間かかり、しかも 3B の LLM モデル1つです。それなのにあなたは Qwen3-14b 0.929、Qwen2.5-14b 0.928、Qwen3-8b 0.927、Qwen3-4b-instruct-2507、llama3.1-8b、Ettin 400M を、200% test + 100% train で12時間以内に学習している、すごい！

あなたのコードがこの public notebook よりはるかに速い理由は分かりますか？ 分からなければ、あなたのコードを見て調べてみます。

WOOSUNG YOON — 質問

多くを学びました。

長さ順のソートと dynamic padding は、速度に大きく影響しますか？

私は 3B か 4B のモデルを2つ学習するだけで、notebook の12時間の実行時間を使い切ってしまったからです。

その鍵となる方法に興味があります。

Guanshuo Xu — 回答

Unsloth は速いですが、はるかに速いかどうかは分かりません。私のコードでは、Qwen3-14b を1枚の T4 でファインチューニングするのに約6時間かかります。3b モデルのファインチューニングに6時間かかるのは不合理に思えます…

私の方法では、次のすべてが速度を改善します。

1. 学習中は1つのトークンだけで loss を計算する。
2. 推論中は forward pass のみを行い、生成はしない。
3. 推論中はテストデータを長さでソートする。

Guanshuo Xu — 回答

長さ順のソートと dynamic padding は速度に大きく影響しますか？ はい、するはずです。

3B/4B モデル2つで12時間は、確かに遅いです。私の notebook を下にスクロールすると、使っていない素の pytorch DDP のコードがあります。そのコードで、4B モデル2つと 3B モデル1つを約3時間で学習・推論できます。

c-number — 質問

1位おめでとうございます！

私も unsloth を使いましたが、transformers を使った場合と比べて、ほぼ毎回 ~0.005 ほど低いスコアになりました。

同じ問題を経験しましたか？ それとも問題はなかったですか？

Guanshuo Xu — 回答

私は問題ありません。unsloth と transformers を使った私の Qwen3-4b のスコアは近いです。

c-number — 質問

> unsloth と transformers を使った私の Qwen3-4b のスコアは近いです。

同じハイパーパラメータ（learning rate など）で同程度のスコアになりましたか？

Guanshuo Xu — 回答

ハイパーパラメータはほぼ同じで、プロンプトが違います。一方は template を使い、もう一方は使っていません。どれだけ頑張っても、Qwen3-4b で達成できる最高は 0.925 です。

c-number — 質問

推論に vllm を使わなかった理由はありますか？ @wowfattie

Guanshuo Xu — 回答

必要なのは単純な forward pass と最後のトークンの logits の出力だけだからです。これは vllm を使わない方が簡単です。プロンプトの長さは短く、モデルサイズもそれほど大きくなく、GPU も1枚なので、vllm を使う利点はないと考え、vllm に時間をかけませんでした。